

Department of Electrical and Information Engineering  
University of Ruhuna

Advanced Artificial Intelligence – Group Project

---

# Cyber AI Agent

Multi-Model Network Intrusion Detection and Explanation  
System

---

Student Name 1   Index No.

Student Name 2   Index No.

Student Name 3   Index No.

Final Report

23 June 2026

**Domain:** Telecommunications / Cybersecurity

**Deliverable:** Functional AI product with reproducible code, notebooks, and evaluation artifacts

# Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                    | <b>1</b>  |
| <b>2</b> | <b>Problem Statement and Motivation</b>     | <b>1</b>  |
| <b>3</b> | <b>Literature Review</b>                    | <b>2</b>  |
| <b>4</b> | <b>Methodology</b>                          | <b>3</b>  |
| 4.1      | Dataset and Feature Space . . . . .         | 3         |
| 4.2      | Training Workflow . . . . .                 | 4         |
| 4.3      | Mapping to Advanced AI Techniques . . . . . | 4         |
| 4.4      | Runtime Detection Logic . . . . .           | 5         |
| <b>5</b> | <b>Implementation Details</b>               | <b>6</b>  |
| 5.1      | System Architecture . . . . .               | 6         |
| 5.2      | Backend Implementation . . . . .            | 6         |
| 5.3      | API and Frontend Product . . . . .          | 7         |
| 5.4      | Reproducibility . . . . .                   | 7         |
| <b>6</b> | <b>Experimental Setup and Results</b>       | <b>7</b>  |
| 6.1      | Experimental Setup . . . . .                | 7         |
| 6.2      | Individual Model Results . . . . .          | 8         |
| 6.3      | Autoencoder Threshold . . . . .             | 8         |
| 6.4      | Fusion and Ablation Results . . . . .       | 9         |
| 6.5      | Training Cost . . . . .                     | 9         |
| <b>7</b> | <b>Evaluation and Performance Analysis</b>  | <b>10</b> |
| 7.1      | Quantitative Evaluation . . . . .           | 10        |
| 7.2      | Qualitative Evaluation . . . . .            | 10        |
| 7.3      | Comparison With Baselines . . . . .         | 10        |

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>8</b>  | <b>Discussion</b>                           | <b>11</b> |
| 8.1       | Strengths . . . . .                         | 11        |
| 8.2       | Limitations . . . . .                       | 11        |
| 8.3       | Recommended Production Refinement . . . . . | 11        |
| <b>9</b>  | <b>Conclusion and Future Work</b>           | <b>12</b> |
| <b>10</b> | <b>References</b>                           | <b>13</b> |
| <b>A</b>  | <b>Appendices</b>                           | <b>13</b> |
| A.1       | Core Project Structure . . . . .            | 13        |
| A.2       | Runtime Commands . . . . .                  | 14        |
| A.3       | Team Contribution Breakdown . . . . .       | 14        |
| A.4       | Submission Checklist . . . . .              | 15        |

## List of Figures

|   |                                                             |   |
|---|-------------------------------------------------------------|---|
| 1 | Runtime architecture of the Cyber AI Agent backend. . . . . | 6 |
|---|-------------------------------------------------------------|---|

## List of Tables

|   |                                                   |    |
|---|---------------------------------------------------|----|
| 1 | Required Runtime Features . . . . .               | 3  |
| 2 | Assignment Technique Mapping . . . . .            | 4  |
| 3 | Backend Components and Responsibilities . . . . . | 6  |
| 4 | Main API Endpoints . . . . .                      | 7  |
| 5 | Saved Individual Model Metrics . . . . .          | 8  |
| 6 | Saved Ensemble Experiment Metrics . . . . .       | 9  |
| 7 | Saved Training Times . . . . .                    | 9  |
| 8 | Team Contribution Template . . . . .              | 14 |

## Executive Summary

Cyber AI Agent is a functional artificial intelligence product for detecting and explaining malicious network traffic from flow-export CSV files. The system targets the telecommunications and cybersecurity domain, where network anomaly detection is operationally important because attacks such as DDoS, brute force activity, port scanning, botnet traffic, infiltration, and web attacks can disrupt services and compromise user data.

The project implements an eight-layer detection pipeline in a FastAPI backend and exposes the results through a React dashboard. The runtime sequence is ingestion, preprocessing, XGBoost classification, BERT-based text classification, autoencoder anomaly detection, fusion, static threat-intelligence enrichment, and LLM-based explanation. This provides a complete demonstrable product rather than only an offline experiment.

The project satisfies and exceeds the assignment requirement to implement at least three Advanced AI techniques. It uses natural language processing by converting network flow records into text, transformer transfer learning through a fine-tuned BERT classifier, generative AI through a denoising autoencoder trained on benign traffic, ensemble learning through a fusion layer, and prompt engineering through a structured LLM explanation layer. The project also includes training notebooks, saved artifacts, a README/setup guide, backend unit tests, and a frontend demo workflow.

The strongest individual model is XGBoost, with a saved test accuracy of 0.9974 and F1-score of 0.9969 over 241,522 test samples. The standalone BERT artifact reports an accuracy of 0.9513 and F1-score of 0.9465, while the autoencoder provides unsupervised anomaly support with a detection-rate-oriented evaluation. The ensemble experiment achieves a fusion F1-score of 0.9713. A key lesson from the evaluation is that the ensemble improves interpretability and model diversity, but XGBoost remains the most reliable decision engine in the saved results. Therefore, the recommended future production hardening is an XGBoost-centric architecture where the autoencoder is a risk overlay and the language model is explanation-only.

## Problem Statement and Motivation

Network intrusion detection systems must process large volumes of traffic logs and identify malicious behavior quickly. In practice, this is difficult for four reasons. First, network-flow data is high-dimensional and noisy. Second, attack classes are imbalanced: benign traffic dominates many datasets, while rare attacks may have very few examples. Third, a single model may be accurate on common classes but weak on abnormal or rare patterns. Fourth, black-box predictions are difficult for security analysts to trust without clear explanations.

The objective of this project is to build a reproducible AI system that can:

1. accept network-flow CSV files through a usable interface;
2. classify each flow as benign or malicious;
3. identify the likely attack type when malicious;
4. flag unusual behavior using unsupervised anomaly detection;
5. enrich suspicious records with static threat-intelligence context;
6. generate analyst-readable explanations and mitigation guidance;
7. provide transparent debug traces for every pipeline layer.

The project is relevant to the Telecommunications domain listed in the assignment brief because service providers continuously monitor network flows for denial-of-service, scanning, brute force, and botnet activity. A working AI-assisted monitoring tool can reduce analyst workload and improve response time.

## Literature Review

Intrusion detection has traditionally used rule-based signatures and supervised machine learning. Signature systems are useful for known patterns, but they struggle with evolving attacks and noisy traffic. Supervised machine learning models such as random forests, support vector machines, and gradient boosting models have therefore been widely used for tabular intrusion datasets. XGBoost is particularly effective for structured data because it builds additive decision trees, handles non-linear interactions, supports regularization, and performs well on heterogeneous numerical features [3].

Transformer-based models changed the state of the art in natural language processing by learning contextual representations from large corpora. BERT introduced bidirectional transformer pre-training and has been adapted to many classification tasks through fine-tuning [1]. Although raw network traffic is numeric, this project converts each flow into a compact textual description so that a transformer can learn semantic patterns such as bursty traffic, payload size, and flow imbalance.

Autoencoders are neural networks trained to reconstruct their input. When trained mainly or only on benign traffic, they learn the normal feature manifold. Inputs that reconstruct poorly can be treated as anomalies. This makes autoencoders valuable in intrusion detection because

they can highlight unusual behavior even when the specific attack class is difficult to identify [6].

Recent applied AI systems increasingly combine multiple models with explanation layers. The motivation is not only to increase predictive performance, but also to improve transparency. This project follows that direction by combining tabular classification, transformer classification, unsupervised anomaly detection, and language-model explanation in one deployable workflow.

## Methodology

### Dataset and Feature Space

The project uses CIC/ISCX-style network flow data exported as CSV files. The runtime backend expects 18 numerical features, listed in Table 1. Extra columns are allowed, and source IP metadata is used when present. If no source IP column exists, the ingestion layer uses unknown placeholders so that standard flow-export files still run.

Table 1: Required Runtime Features

| Feature group       | Features                                                                                                           |
|---------------------|--------------------------------------------------------------------------------------------------------------------|
| Duration and volume | Flow Duration; Total Fwd Packets; Total Backward Packets; Total Length of Fwd Packets; Total Length of Bwd Packets |
| Rate features       | Flow Bytes/s; Flow Packets/s                                                                                       |
| Packet length       | Fwd Packet Length Mean; Bwd Packet Length Mean; Average Packet Size                                                |
| Inter-arrival time  | Flow IAT Mean; Fwd IAT Mean; Bwd IAT Mean                                                                          |
| TCP flags and port  | Fwd PSH Flags; Bwd PSH Flags; Fwd URG Flags; Bwd URG Flags; Destination Port                                       |

The label set contains 15 classes: BENIGN, Bot, DDoS, four DoS variants, FTP-Patator, SSH-Patator, Heartbleed, Infiltration, PortScan, and three web attack classes.

## Training Workflow

The training workflow is implemented as five Jupyter notebooks:

1. `01_preprocessing.ipynb`: loads and combines flow files, cleans data, selects the 18 features, scales the features, creates train/validation/test splits, and saves preprocessing artifacts.
2. `02_xgboost.ipynb`: trains the supervised XGBoost classifier and saves the model and metrics.
3. `03_bert.ipynb`: generates text from flow features, fine-tunes the BERT classifier, and saves the transformer model directory and metrics.
4. `04_autoencoder.ipynb`: trains a denoising autoencoder on benign traffic, tunes the reconstruction threshold, and saves the model and threshold.
5. `05_ensemble.ipynb`: loads all trained models, performs fusion experiments, runs ablation studies, and exports ensemble metrics and plots.

This notebook separation improves reproducibility because each model can be retrained and evaluated independently before being integrated into the backend.

## Mapping to Advanced AI Techniques

Table 2: Assignment Technique Mapping

| Assignment technique        | Project implementation                                                                                                                                                |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Natural Language Processing | The preprocessing layer converts numeric flow records into natural-language descriptions such as packet count, traffic rate, payload size, and flow-balance behavior. |
| Transformer-based models    | The BERT layer uses prajjwal1/bert-tiny fine-tuned for 15 traffic classes.                                                                                            |
| Generative AI               | The denoising autoencoder reconstructs benign flows and flags high reconstruction error as anomalous behavior.                                                        |
| Transfer learning           | The BERT model starts from pretrained transformer weights and is adapted to the intrusion-detection class set.                                                        |
| Prompt engineering          | The LLM layer uses a strict JSON prompt with cybersecurity-specific rules to generate explanation, risk, mitigation, severity, and model-agreement fields.            |
| Ensemble learning           | The fusion layer combines supervised and unsupervised model outputs into a single threat list with severity and confidence fields.                                    |

## Runtime Detection Logic

At runtime, the FastAPI backend executes an eight-layer pipeline:

1. **Ingestion:** parse CSV, strip column whitespace, validate all required columns, and extract IPs where available.
2. **Preprocessing:** align feature order, replace infinite values, fill missing values, apply the saved scaler, and generate NLP text.
3. **XGBoost:** classify each scaled flow using the trained tabular model.
4. **BERT:** classify each generated text description using the trained transformer classifier.
5. **Autoencoder:** reconstruct each scaled feature vector and compute reconstruction error and anomaly score.
6. **Fusion:** combine XGBoost, BERT, and autoencoder outputs. The current backend uses weights 0.40, 0.40, and 0.20 for XGBoost, BERT, and the autoencoder respectively. A row is treated as a threat if any detector flags attack/anomaly; when XGBoost and BERT disagree on the attack type, the higher-confidence supervised model determines the label.
7. **Threat-intelligence enrichment:** add static IP reputation and CVE mapping from local lookup tables. This is a coursework demonstration layer, not a live external feed.
8. **LLM explanation:** generate or mock structured analyst explanations for the top threats.

The current implementation is therefore a multi-model fusion system. Based on the saved metrics, a recommended future production refinement is to make XGBoost the sole final classifier, use the autoencoder only as a severity/risk overlay, and reserve language models for explanation only. This refinement is discussed in Section 8; it is not overstated as the current runtime design.

## Implementation Details

### System Architecture

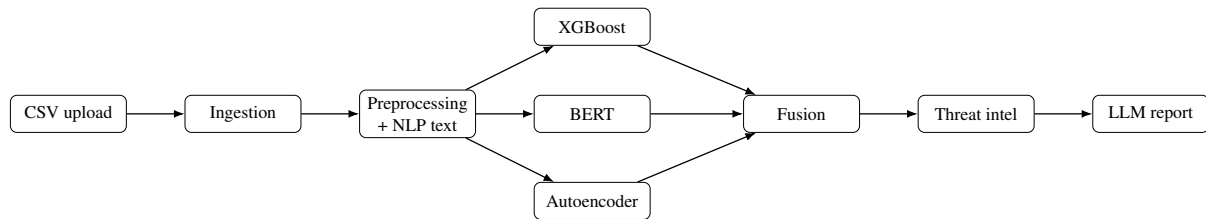


Figure 1: Runtime architecture of the Cyber AI Agent backend.

### Backend Implementation

The backend is implemented in Python using FastAPI. The main application is defined in `backend/app/main.py`; the pipeline orchestration is in `backend/app/pipeline.py`. Each AI step is isolated in `backend/app/layers/`, which makes the system easier to test and debug.

Table 3: Backend Components and Responsibilities

| File / module                     | Responsibility                                                                                                                     |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>main.py</code>              | Creates the FastAPI application, enables CORS, loads environment variables, and registers API routers.                             |
| <code>pipeline.py</code>          | Loads singleton layer objects, executes all eight layers, serializes debug outputs, and returns final results.                     |
| <code>ingestion.py</code>         | Parses uploaded CSV content and validates the 18 required features.                                                                |
| <code>preprocessing.py</code>     | Reindexes features, cleans invalid values, applies the saved scaler, and generates text descriptions.                              |
| <code>xgboost_model.py</code>     | Loads <code>xgboost_model.pkl</code> and returns labels, attack names, and probabilities.                                          |
| <code>bert_model.py</code>        | Loads the local BERT classifier and tokenizes generated texts for inference.                                                       |
| <code>autoencoder_model.py</code> | Loads <code>autoencoder.keras</code> and <code>autoencoder_threshold.json</code> ; returns reconstruction error and anomaly flags. |
| <code>fusion.py</code>            | Combines model outputs, resolves labels, calculates fused risk score, and creates the threat list.                                 |
| <code>mcp_tools.py</code>         | Adds local IP reputation and CVE mapping to detected threats.                                                                      |
| <code>llm_explainer.py</code>     | Produces structured JSON explanations using Groq or a deterministic mock fallback.                                                 |

## API and Frontend Product

The product includes a React + Vite frontend. The demo workflow begins on the upload page, where the user selects a CSV file. The frontend sends the file to `/api/upload-logs`, then triggers `/api/predict?debug=true`. After prediction, the dashboard displays summary statistics, threat severity distribution, attack type distribution, per-model breakdown, LLM reports, and layer-by-layer debug telemetry.

Table 4: Main API Endpoints

| Method | Endpoint                | Purpose                                                    |
|--------|-------------------------|------------------------------------------------------------|
| GET    | /                       | Health check and documentation pointer.                    |
| POST   | /api/upload-logs        | Upload a CSV log file.                                     |
| POST   | /api/predict            | Run the full pipeline on an uploaded file.                 |
| GET    | /api/dashboard          | Return latest summary, threat counts, and model breakdown. |
| GET    | /api/reports            | Return enriched LLM threat reports.                        |
| GET    | /api/debug/runs         | List saved debug run IDs.                                  |
| GET    | /api/debug/run/{run_id} | Load a specific debug trace.                               |
| GET    | /api/debug/latest       | Load the latest debug trace.                               |

## Reproducibility

The repository includes `backend/requirements.txt` for Python dependencies, `frontend/package.json` for the React frontend, setup instructions in the README and `SETUP.md`, and a `backend/.env.example` file for optional LLM configuration. Runtime model artifacts are loaded from `backend/trained_models/`. The backend also writes JSON debug traces to `backend/debug_logs/`, allowing intermediate outputs to be inspected after each run.

## Experimental Setup and Results

### Experimental Setup

The dataset was split into training, validation, and test sets during preprocessing. XGBoost and BERT were evaluated as multiclass classifiers. The autoencoder was trained on benign traffic and evaluated as an anomaly detector using reconstruction error. The ensemble notebook then loaded all saved artifacts and compared individual and fused model behavior over the test split.

Metrics used:

- **Accuracy:** proportion of all correctly classified records.
- **Precision:** proportion of predicted attacks that are correct.
- **Recall:** proportion of actual attacks detected.
- **F1-score:** harmonic mean of precision and recall.
- **False positive rate:** benign records incorrectly flagged by the anomaly detector.
- **ROC-AUC:** separability measure for XGBoost probability outputs.

## Individual Model Results

Table 5 reports the saved metrics from the individual model artifact files. These are the strongest direct indicators of each model’s standalone performance.

Table 5: Saved Individual Model Metrics

| Model       | Test samples | Accuracy | Precision | Recall | F1-score |
|-------------|--------------|----------|-----------|--------|----------|
| XGBoost     | 241,522      | 0.9974   | 0.9973    | 0.9974 | 0.9969   |
| BERT        | 91,484       | 0.9513   | 0.9424    | 0.9513 | 0.9465   |
| Autoencoder | 241,522      | 0.8853   | 0.7074    | 0.5726 | 0.6329   |

XGBoost produced the best standalone supervised result. This is expected because the dataset consists mainly of structured flow features, which are well suited to gradient-boosted decision trees. BERT also performed strongly, showing that the generated text representation preserved useful traffic behavior. The autoencoder had lower F1-score, but it serves a different purpose: identifying unusual behavior through reconstruction error rather than assigning an exact attack class.

## Autoencoder Threshold

The saved autoencoder threshold is 0.0051889612, selected as the 95th percentile threshold. The validation F1-score at this threshold is 0.6302, and the test false positive rate is 0.0494. This supports its role as a supplementary anomaly detector rather than the only decision engine.

## Fusion and Ablation Results

The ensemble experiment in `05_ensemble.ipynb` saved the results in Table 6. This notebook performs a separate combined inference run, so its individual model comparison values can differ from the standalone notebook files due to evaluation sample size and pipeline context.

Table 6: Saved Ensemble Experiment Metrics

| Component                   | Accuracy | F1-score |
|-----------------------------|----------|----------|
| XGBoost in ensemble run     | 0.9969   | 0.9964   |
| BERT in ensemble run        | 0.8018   | 0.7364   |
| Autoencoder in ensemble run | –        | 0.6646   |
| Fusion ensemble             | 0.9750   | 0.9713   |

The fusion result improves substantially over the weaker BERT and autoencoder signals in the ensemble experiment, but it does not exceed XGBoost. This is an important finding: multi-model fusion improves transparency and provides complementary evidence, but the current saved metrics show that XGBoost should be treated as the most reliable classifier for a production design.

## Training Cost

Table 7: Saved Training Times

| Model       | Training time (seconds) | Approximate time |
|-------------|-------------------------|------------------|
| XGBoost     | 1788.15                 | 29.8 minutes     |
| BERT        | 12968.83                | 3.6 hours        |
| Autoencoder | 7427.44                 | 2.1 hours        |

BERT and the autoencoder require significantly more training time than XGBoost. This supports the practical decision to use XGBoost as the most important runtime classifier and to use the other models for complementary analysis and explanation.

## Evaluation and Performance Analysis

### Quantitative Evaluation

From the saved metrics, XGBoost achieved the best balance of accuracy, recall, and F1-score. Its ROC-AUC value of 0.9920 further indicates strong probability separation. BERT's standalone metrics are also strong, but its ensemble-run metrics were weaker, suggesting that text-generation consistency, evaluation sample differences, or artifact alignment must be controlled carefully in future experiments. The autoencoder had a moderate F1-score but a low false positive rate of approximately 4.94%, which is useful for prioritizing suspicious behavior.

### Qualitative Evaluation

The qualitative strength of the system is interpretability. The dashboard does not only show a final label; it exposes XGBoost output, BERT output, autoencoder anomaly score, fused risk, severity, static threat-intelligence context, and a natural-language explanation. The debug page also allows layer-by-layer inspection, which is useful for identifying mismatches between training artifacts and runtime behavior.

### Comparison With Baselines

A single XGBoost model is the strongest performance baseline in this project. The final system adds value beyond this baseline by providing:

- an independent transformer perspective from textual traffic descriptions;
- unsupervised anomaly evidence from reconstruction error;
- static IP/CVE enrichment for analyst context;
- LLM or mock explanations for readable reports;
- frontend visualization and debug traceability.

Therefore, the project should not be judged only as a marginal F1-score optimization. It is a full AI product that combines detection, explanation, and demonstration usability.

## Discussion

### Strengths

The project is strong in four areas. First, it implements more than the minimum number of required Advanced AI techniques. Second, it is end-to-end: the system includes training notebooks, saved artifacts, an API, and a frontend. Third, the pipeline is modular, so each layer can be tested or replaced. Fourth, the dashboard and debug route make model behavior visible, which improves trust and supports demonstration.

### Limitations

The current backend fusion logic allows any of the three detectors to mark a record as a threat. This improves sensitivity but can also increase inconsistency when one model is overconfident or misaligned. The saved ensemble metrics also show that fusion does not outperform XGBoost. Therefore, the current system is best described as an explainable multi-model research prototype rather than a fully calibrated production SOC detector.

The static threat-intelligence layer is intentionally limited. It uses a small local blacklist and CVE map, not a live reputation service. This is appropriate for coursework demonstration but should not be described as real-time threat intelligence.

The LLM layer depends on a Groq API key for live explanations. When no key is configured, the system uses mock explanations. This keeps the product demonstrable, but live LLM behavior should be evaluated separately for reliability and factual safety.

### Recommended Production Refinement

The saved results support a future XGBoost-centric production design. In that design, XGBoost would be the only final attack classifier, the autoencoder would only raise or lower risk based on anomaly score, and the language model would be used only after the decision for explanation. This avoids conflicting votes and prevents BERT or anomaly-only outputs from overriding the strongest classifier.

A production decision rule could be:

```
1 pred = xgb_model.predict(X)
2 is_anomaly = autoencoder_score(X) > threshold
3
4 if pred != "BENIGN":
5     label = pred
```

```
6     severity = "HIGH" if is_anomaly else "MEDIUM"
7 else:
8     label = "BENIGN"
9     severity = "LOW" if is_anomaly else "VERY_LOW"
```

Listing 1: Recommended future production decision rule

This refinement should be described as future work unless the backend code is changed to match it.

## Conclusion and Future Work

Cyber AI Agent successfully demonstrates a complete Advanced Artificial Intelligence project in the cybersecurity domain. It combines NLP, transformer transfer learning, unsupervised autoencoder anomaly detection, ensemble fusion, prompt-engineered explanation, a FastAPI backend, and a React dashboard. The project therefore addresses all major assignment requirements: real-world relevance, multiple AI techniques, model implementation, evaluation, report quality, and working demo functionality.

The evaluation shows that XGBoost is the strongest classifier for the structured flow dataset, while BERT and the autoencoder add complementary interpretability and anomaly evidence. The final product is suitable for a coursework demonstration and provides a clear foundation for future hardening.

Future work should focus on:

1. implementing the XGBoost-centric production decision rule;
2. calibrating severity thresholds on a validation set;
3. replacing static IP/CVE lookup tables with live threat-intelligence APIs;
4. adding model and data versioning for reproducibility;
5. expanding frontend reporting to export PDF/CSV incident summaries;
6. adding continuous integration so backend tests and frontend builds run automatically.

## References

## References

- [1] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 4171–4186.
- [2] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [5] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 108–116.
- [6] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- [7] Ramirez, S. (2024). *FastAPI documentation*. <https://fastapi.tiangolo.com/>

## Appendices

### Core Project Structure

```

1 cyber-ai-agent /
2 |-- backend/
3 |   |-- app/
4 |   |   |-- main.py
5 |   |   |-- pipeline.py

```

```

6 |   |   |-- layers/
7 |   |   |-- routes/
8 |   |   |-- models/
9 |   |   +-- utils/
10 |   |-- notebooks/
11 |   |-- tests/
12 |   |-- trained_models/
13 |   |-- requirements.txt
14 |   +-- init_models.py
15 |-- frontend/
16 |   |-- src/
17 |   |-- package.json
18 |   +-- vite.config.js
19 |-- reports/
20 |-- README.md
21 +-- SETUP.md

```

Listing 2: Core project structure

## Runtime Commands

```

1 cd backend
2 .\venv\Scripts\Activate.ps1
3 uvicorn app.main:app --reload --port 8000
4
5 cd frontend
6 npm install
7 npm run dev

```

Listing 3: Backend and frontend demo commands

## Team Contribution Breakdown

Table 8: Team Contribution Template

| Member         | Contribution                                                           |
|----------------|------------------------------------------------------------------------|
| Student Name 1 | Data preprocessing, XGBoost training, metric analysis.                 |
| Student Name 2 | BERT training, autoencoder training, notebook documentation.           |
| Student Name 3 | FastAPI backend, React dashboard, report and presentation preparation. |

## Submission Checklist

- Final report uses required sections, 12 pt font, 1.5 spacing, 1 inch margins, page numbers, and references.
- README and SETUP guide describe dependencies and reproducible execution.
- Training notebooks contain markdown explanations and save artifacts.
- Backend can run on port 8000 and frontend on port 3000.
- Video presentation should cover problem, approach, results, and live dashboard demo within five minutes.